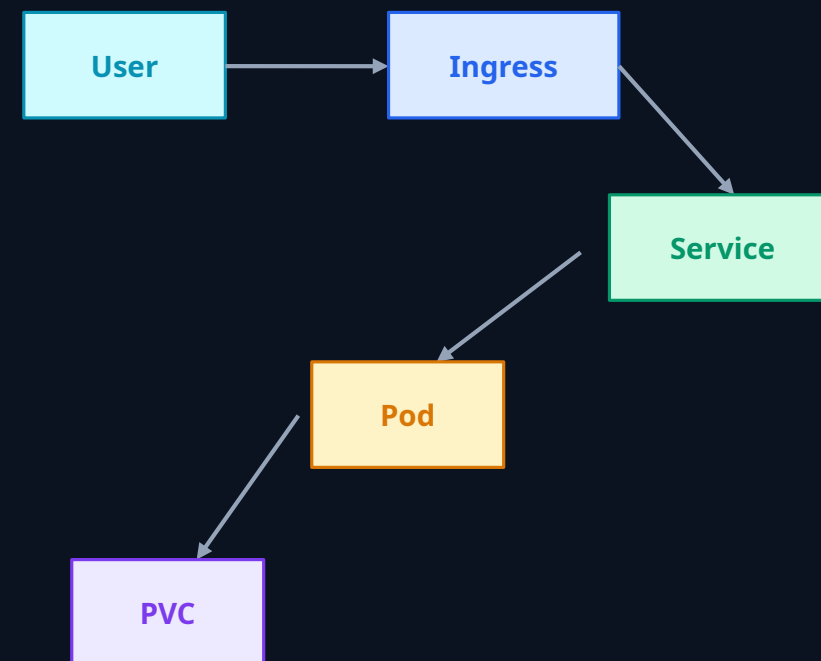


Kubernetes Production Troubleshooting Carousel

Common production faults, what to check first, how to repair, and how to verify recovery.



The production troubleshooting ladder

K8s prod runbook · 02

Always move from outside to inside, then from desired state to actual state.

1. Client / DNS

Can user resolve and reach the entry?

2. LB / Ingress

Is host/path/TLS routed?

3. Service

Does selector find endpoints?

4. Pod

Ready? restarting? image? probes?

5. Container app

Port, bind address, logs, exit code

6. Node / storage

Resources, taints, PVC, volume permissions

```
kubectl get nodes,pods,svc,ingress,pvc -A
kubectl describe <object> <name> -n <ns>
kubectl logs <pod> -n <ns> --previous
kubectl get events -n <ns> --sort-by=.lastTimestamp
```

Pattern: fault → investigate in order → root cause → repair → verify recovery → keep the lesson

Golden command map

Use these commands before changing YAML. They show facts, events, logs, ownership, and endpoints.

Inventory

```
kubectl get all -n app
kubectl get pods -o wide
kubectl get deploy,rs,svc,ep
```

Explain status

```
kubectl describe pod POD
kubectl get events --sort-by=.lastTimestamp
```

Logs

```
kubectl logs POD -c app
kubectl logs POD --previous
```

Network

```
kubectl get svc,endpoints
kubectl exec POD -- nslookup svc
kubectl port-forward svc/api 8080:80
```

Rollout

```
kubectl rollout status deploy/api
kubectl rollout history deploy/api
kubectl rollout undo deploy/api
```

Resources

```
kubectl top pods,nodes
kubectl describe node NODE
kubectl get hpa,pvc,sc
```

Use one repeatable incident template

This keeps troubleshooting calm during production pressure.

1 Fault

What is failing? error, status, user impact.

2 Scope

One pod, one namespace, one node, or all cluster?

3 Evidence

Describe, events, logs, endpoints, metrics.

4 Change

What changed recently: image, config, rollout, node, cert, quota?

5 Fix

Apply the smallest safe repair.

6 Verify

Ready, traffic works, logs clean, no new events.

```
# tiny report format
Fault: ...
Investigation: commands + output
Root cause: ...
Repair: ...
Verification: ...
Prevention / lesson: ...
```

Pattern: fault → investigate in order → root cause → repair → verify recovery → keep the lesson

Image cannot be downloaded

ImagePullBackOff / ErrImagePull

What breaks visually



Production symptom

**Pod is stuck in ImagePullBackOff and never starts.
Read events first. Do not guess from pod status only.**

```
kubectl describe pod web-abc -n app
kubectl get events -n app --sort-by=.lastTimestamp
kubectl get pod web-abc -o jsonpath="{.spec.containers[*].image}"
kubectl get secret -n app | grep regcred
```

1 Fault signal
ImagePullBackOff, ErrImagePull, 401, 403, 404, or timeout in Events.

2 Investigate in order
Describe pod → read Events → check image name/tag → check registry secret/network.

3 Root cause
Bad image tag, private registry auth missing, image removed, or node cannot reach registry.

4 Repair
Fix image name/tag, create imagePullSecret, attach it to ServiceAccount or Pod, then restart rollout.

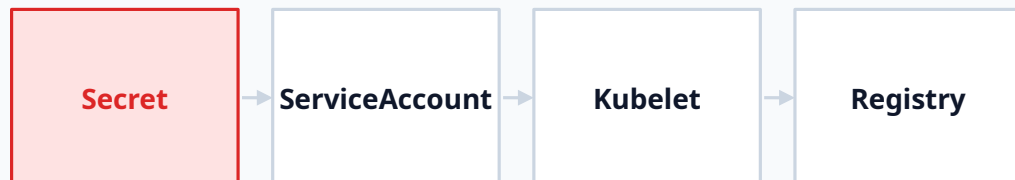
5 Verify recovery
Pod moves Pulling → Pulled → Created → Started → Running/Ready.

6 Lesson
Pin images carefully and test registry access from nodes before production rollout.

Private registry auth is missing

Image pull works locally but not in cluster

What breaks visually



Production symptom

**The image is real, but kubelet has no valid credentials.
Read events first. Do not guess from pod status only.**

```
kubectl create secret docker-registry regcred --docker-server=REG --docker-username=USER --docker-password=PASS -n app
kubectl patch serviceaccount default -n app -p '{"imagePullSecrets":[{"name":"regcred"}]}'
kubectl rollout restart deploy/api -n app
```

1 Fault signal

Events show unauthorized, denied, or pull access denied.

2 Investigate in order

Check imagePullSecrets, ServiceAccount, namespace, registry URL, and token expiry.

3 Root cause

Secret exists in wrong namespace, wrong registry server value, or ServiceAccount not using it.

4 Repair

Recreate registry secret in same namespace and link it to ServiceAccount or Pod spec.

5 Verify recovery

Events show Successfully pulled image; new ReplicaSet becomes available.

6 Lesson

Registry secrets are namespace-scoped. Same secret name in another namespace does not help.

Wrong executable or entrypoint

Container cannot start command

What breaks visually



Production symptom

**Container fails immediately with exec format or file not found.
Read events first. Do not guess from pod status only.**

```
kubectl describe pod api-abc -n app
kubectl logs api-abc -n app
kubectl get pod api-abc -o yaml | grep -A8 "command:"
docker run --rm IMAGE sh -c "ls -l /app"
```

1 Fault signal

Errors like "executable file not found", permission denied, or exec format error.

2 Investigate in order

Check image OS/CPU arch, ENTRYPOINT, command/args, file path, and execute bit.

3 Root cause

YAML overrides Dockerfile command incorrectly or binary/script is missing/not executable.

4 Repair

Correct command/args, chmod +x script, use correct shell path, or build image for node architecture.

5 Verify recovery

Container starts and stays running; logs show app boot message.

6 Lesson

Do not blindly override command/args. Know Dockerfile ENTRYPOINT vs Kubernetes command.

Application exits repeatedly

CrashLoopBackOff

K8s prod runbook · 08

What breaks visually



Production symptom

**Pod starts, exits, restarts, then back-off delay increases.
Read events first. Do not guess from pod status only.**

```
kubectl get pod api-abc -n app
kubectl describe pod api-abc -n app
kubectl logs api-abc -n app --previous
kubectl get pod api-abc -o jsonpath="{.status.containerStatuses[0].lastState}"
```

1 Fault signal
STATUS=CrashLoopBackOff, RESTARTS keeps increasing.

2 Investigate in order
Read previous logs → exit code → env/config/secrets → dependency reachability.

3 Root cause
App process exits due to missing config, bad DB URL, bad command, or unhandled exception.

4 Repair
Fix config, dependency, command, or image; deploy a new ReplicaSet.

5 Verify recovery
Restart count stops increasing and pod remains Ready through several probe periods.

6 Lesson
CrashLoopBackOff is a symptom. Previous logs usually show the real failure.

Referenced ConfigMap does not exist

CreateContainerConfigError

What breaks visually



Production symptom

Pod is created but container never starts because config object is missing.

Read events first. Do not guess from pod status only.

```
kubectl describe pod web-abc -n app
kubectl get cm -n app
kubectl get deploy web -o yaml | grep -A8 configMapRef
kubectl create configmap app-config --from-literal=APP_MODE=prod -n app
```

1 Fault signal
Pod status CreateContainerConfigError and Events mention configmap not found.

2 Investigate in order
Check namespace, ConfigMap name, volume/envFrom references, and Helm/Kustomize output.

3 Root cause
Deployment references a ConfigMap that was not created or was created in another namespace.

4 Repair
Create the ConfigMap with expected name/keys or update Deployment reference.

5 Verify recovery
Pod changes from CreateContainerConfigError to Running/Ready after restart.

6 Lesson
Apply config objects before workloads; use the same namespace for workload and ConfigMap.

Secret exists but key is wrong

Secret name is correct but env/volume key fails

What breaks visually



Production symptom

Kubernetes finds the Secret, but the requested key is missing or wrong.

Read events first. Do not guess from pod status only.

```
kubectl describe pod api-abc -n app
kubectl get secret db-secret -n app -o yaml
kubectl get deploy api -o yaml | grep -A12 secretKeyRef
kubectl create secret generic db-secret --from-literal=DB_PASSWORD=xxx -n app
--dry-run=client -o yaml | kubectl apply -f -
```

1 Fault signal
Events say key not found, or app starts with empty/wrong password.

2 Investigate in order
Compare exact key spelling in secretKeyRef with keys stored in Secret data.

3 Root cause
Secret contains password but Deployment expects DB_PASSWORD, PASSWORD, or another key name.

4 Repair
Patch Deployment to correct key or recreate Secret with the expected key.

5 Verify recovery
Pod starts; app authenticates to dependency; no key-related Events remain.

6 Lesson
Secret name and key are different. Always verify both.

Running Pod is not Ready

App runs but Service will not send traffic

What breaks visually



Production symptom
Pod STATUS is Running, but READY shows 0/1 or endpoints are empty.
Read events first. Do not guess from pod status only.

```
kubectl get pod -n app
kubectl describe pod api-abc -n app
kubectl get endpoints api-svc -n app
kubectl logs api-abc -n app
```

1 Fault signal
READY 0/1, readiness probe failed, service endpoints missing.

2 Investigate in order
Describe pod → readiness Events → curl probe path inside pod → check app port.

3 Root cause
Readiness probe path/port is wrong or app is running but not ready for traffic.

4 Repair
Fix readiness path/port/initialDelay, or fix app health endpoint/dependency.

5 Verify recovery
READY 1/1 and Service endpoints include pod IP.

6 Lesson
Running means process exists. Ready means traffic should be sent.

Liveness causes unnecessary restarts

Bad liveness probe kills healthy app

What breaks visually



Production symptom

App is healthy enough to serve, but kubelet keeps restarting it. Read events first. Do not guess from pod status only.

```
kubectl describe pod api-abc -n app
kubectl logs api-abc -n app --previous
kubectl get pod api-abc -o yaml | grep -A14 livenessProbe
kubectl patch deploy api --type=json
-p='[{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/initialDelaySeconds","value":60}]'
```

1 Fault signal
Restart count rises and Events show failed liveness probe.

2 Investigate in order
Check liveness endpoint, timeout, period, failureThreshold, and app GC/CPU spikes.

3 Root cause
Liveness probe is too strict, points to dependency-heavy endpoint, or checks wrong port/path.

4 Repair
Use a simple self-check endpoint; increase timeout/failureThreshold; add startupProbe.

5 Verify recovery
No new restarts while readiness still protects traffic.

6 Lesson
Liveness should answer: should this container be restarted? Not: is every dependency perfect?

Slow startup exceeds probe budget

App is killed before it has time to boot

What breaks visually



Production symptom

Heavy app, migrations, or cold cache cause restarts during boot. Read events first. Do not guess from pod status only.

```
kubectl describe pod api-abc -n app
kubectl logs api-abc -n app --previous
kubectl get deploy api -o yaml | grep -A18 startupProbe
kubectl rollout restart deploy/api -n app
```

1 Fault signal

Container restarts before app finishes startup; logs show boot sequence interrupted.

2 Investigate in order

Compare app boot time with $\text{initialDelay} + \text{failureThreshold} \times \text{periodSeconds}$.

3 Root cause

Probe budget is smaller than real startup time.

4 Repair

Add startupProbe or increase initialDelay/failureThreshold for slow services.

5 Verify recovery

Container survives boot, then readiness becomes true.

6 Lesson

Use startupProbe for slow apps so liveness starts only after boot succeeds.

Memory limit triggers OOMKilled

Container uses more RAM than its limit

What breaks visually



Production symptom

**Pod restarts with reason OOMKilled and exit code 137.
Read events first. Do not guess from pod status only.**

```
kubectl describe pod api-abc -n app
kubectl top pod api-abc -n app
kubectl get pod api-abc -o
jsonpath="{.status.containerStatuses[0].lastState.terminated.reason}"
kubectl describe node NODE | grep -A5 -i memory
```

1 Fault signal

Last state reason OOMKilled, exit code 137, sudden restarts under load.

2 Investigate in order

Check limits, actual memory with kubectl top, app heap settings, traffic spike, and node pressure.

3 Root cause

Memory limit is too low or app memory leak/heap exceeds container limit.

4 Repair

Tune app memory, fix leak, raise limit carefully, or scale out.

5 Verify recovery

No OOMKilled events after load test; memory stays below limit.

6 Lesson

Limit is a hard kill line. Request is scheduling guarantee; limit is enforcement.

No node has enough requested CPU

Pod is Pending due to scheduling failure

What breaks visually



Production symptom

**The pod cannot schedule even though the cluster looks alive.
Read events first. Do not guess from pod status only.**

```
kubectl describe pod api-abc -n app
kubectl get nodes
kubectl describe node NODE | grep -A6 "Allocated resources"
kubectl top nodes
```

1 Fault signal
Pod Pending; Events show Insufficient cpu or memory.

2 Investigate in order
Compare pod requests with node allocatable and existing allocated resources.

3 Root cause
Requested CPU/memory is greater than available node capacity.

4 Repair
Reduce requests if safe, add nodes, scale down unused workloads, or use cluster autoscaler.

5 Verify recovery
Pod is assigned to a node and transitions to Running.

6 Lesson
Scheduler uses requests, not current usage, to decide placement.

Node selector matches no nodes

Scheduling blocked by labels

What breaks visually



Production symptom

**Pod stays Pending because no node has the requested label.
Read events first. Do not guess from pod status only.**

```
kubectl describe pod web-abc -n app
kubectl get nodes --show-labels
kubectl get pod web-abc -o yaml | grep -A5 nodeSelector
kubectl label node NODE disk=ssd
```

1 Fault signal
Events: node(s) did not match Pod node affinity/selector.

2 Investigate in order
Check nodeSelector, nodeAffinity, node labels, and spelling/case.

3 Root cause
Workload requires a label that no node currently has.

4 Repair
Add correct label to intended node or change selector/affinity.

5 Verify recovery
Pod schedules onto the expected node.

6 Lesson
Selectors are exact matching rules. One wrong character can block scheduling.

Taints block scheduling

Node requires toleration

What breaks visually



Production symptom

Nodes exist but pod cannot schedule due to taints.

Read events first. Do not guess from pod status only.

```
kubectl describe pod api-abc -n app
kubectl describe node NODE | grep -i taints
kubectl get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
kubectl taint nodes NODE dedicated=apps:NoSchedule
```

1 Fault signal
Events mention untolerated taint.

2 Investigate in order
Check node taints and pod tolerations; confirm app should run there.

3 Root cause
Node is protected by NoSchedule taint and pod has no matching toleration.

4 Repair
Add a toleration to Pod or remove taint if it was accidental.

5 Verify recovery
Pod schedules only where intended.

6 Lesson
Taints repel pods; tolerations allow but do not force scheduling.

Init container prevents app startup

Main app waits forever

What breaks visually



Production symptom

Pod is stuck in Init:0/1 or Init:CrashLoopBackOff.

Read events first. Do not guess from pod status only.

```
kubectl get pod api-abc -n app
kubectl describe pod api-abc -n app
kubectl logs api-abc -c init-db -n app
kubectl get pod api-abc -o yaml | grep -A25 initContainers
```

1 Fault signal
Pod phase shows Init, main container never starts.

2 Investigate in order
Check init container logs, command, secret/config, permissions, dependency DNS.

3 Root cause
Init task fails: migration, wait script, download, chmod, or dependency check.

4 Repair
Fix init command/config/dependency or remove unsafe wait loop.

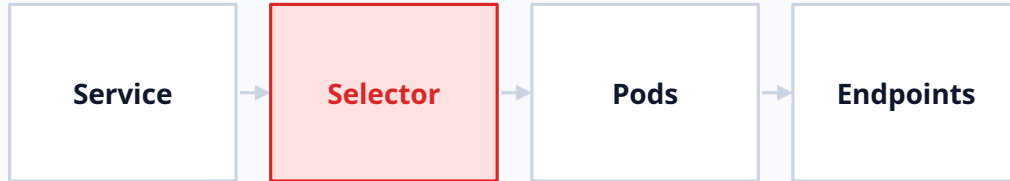
5 Verify recovery
Init completes, main container starts, pod becomes Ready.

6 Lesson
Init containers are gates. If the gate never finishes, the app never starts.

Service selects no Pods

Service has no endpoints

What breaks visually



Production symptom

Service exists but traffic has nowhere to go.

Read events first. Do not guess from pod status only.

```
kubectl get svc api-svc -n app -o yaml
kubectl get pods -n app --show-labels
kubectl get endpoints api-svc -n app
kubectl describe svc api-svc -n app
```

1 Fault signal
Endpoints list is empty; curl service IP times out/refuses.

2 Investigate in order
Compare Service selector with Pod labels; check namespace and readiness.

3 Root cause
Selector labels do not match pod labels or pods are not Ready.

4 Repair
Fix service selector or pod labels; fix readiness if endpoints are gated out.

5 Verify recovery
kubectl get endpoints shows pod IP:port entries.

6 Lesson
Service does not search by pod name. It only selects by labels.

Service forwards to the wrong port

port, targetPort, containerPort mismatch

What breaks visually



Production symptom

**Service has endpoints but traffic fails due to port mismatch.
Read events first. Do not guess from pod status only.**

```
kubectl describe svc api-svc -n app
kubectl get pod api-abc -o yaml | grep -A8 ports
kubectl exec -n app api-abc -- ss -lntp
kubectl port-forward -n app pod/api-abc 8080:8080
```

1 Fault signal
Service reachable but app returns connection refused or timeout.

2 Investigate in order
Check service port, targetPort, named ports, and app listening port.

3 Root cause
Service sends traffic to a port where container is not listening.

4 Repair
Set targetPort to container listening port or fix app containerPort/named port.

5 Verify recovery
Curl ClusterIP:port and Service DNS succeed.

6 Lesson
containerPort is documentation; targetPort decides where Service forwards traffic.

Application binds only to loopback

App listens on 127.0.0.1 instead of 0.0.0.0

What breaks visually



Production symptom

**Health works inside app process, but other pods cannot connect.
Read events first. Do not guess from pod status only.**

```
kubectl exec -n app api-abc -- ss -lntp
kubectl exec -n app api-abc -- curl -s localhost:8080/health
kubectl run tmp --rm -it --image=busybox:1.36 -n app -- wget -qO- http://api-
svc:80/health
```

1 Fault signal
Inside container localhost works, but Service/PodIP from another pod fails.

2 Investigate in order
Check listen address from ss/netstat and app startup config.

3 Root cause
App binds to 127.0.0.1, not to 0.0.0.0 or pod interface.

4 Repair
Configure app host/bind address to 0.0.0.0.

5 Verify recovery
Another pod can curl service DNS successfully.

6 Lesson
In containers, localhost means inside the same container, not the whole pod or cluster.

Client uses an invalid DNS server

DNS resolution fails inside pod

What breaks visually



Production symptom

Application cannot resolve service names even though services exist.

Read events first. Do not guess from pod status only.

```
kubectl exec -n app api-abc -- cat /etc/resolv.conf
kubectl exec -n app api-abc -- nslookup kubernetes.default
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=80
```

1 Fault signal
nslookup fails, app errors show no such host or DNS timeout.

2 Investigate in order
Check pod dnsPolicy, dnsConfig, /etc/resolv.conf, CoreDNS pods, and NetworkPolicy.

3 Root cause
dnsPolicy/dnsConfig overrides cluster DNS or CoreDNS is unhealthy/unreachable.

4 Repair
Use ClusterFirst, remove bad dnsConfig, fix CoreDNS or network rules.

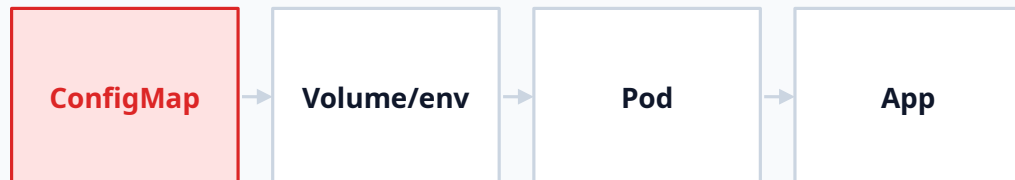
5 Verify recovery
Pod resolves service.namespace.svc.cluster.local and app dependency hostnames.

6 Lesson
Do not hardcode external DNS for pods unless you understand cluster DNS impact.

ConfigMap updated but app sees old value

Runtime config does not refresh automatically

What breaks visually



Production symptom

**The ConfigMap was changed, but the app still uses old data.
Read events first. Do not guess from pod status only.**

```
kubectl get cm app-config -n app -o yaml
kubectl exec -n app api-abc -- cat /config/app.properties
kubectl rollout restart deploy/api -n app
kubectl describe pod api-abc -n app
```

1 Fault signal
kubectl shows new ConfigMap but app behavior remains old.

2 Investigate in order
Check whether config is consumed as env var, volume file, or app cache.

3 Root cause
Env var values are fixed at pod start; mounted files update eventually but app may not reload them.

4 Repair
Restart rollout for env vars; reload app if it watches files; use checksum annotation in GitOps.

5 Verify recovery
New pod contains new env/file value and behavior changes.

6 Lesson
Updating a ConfigMap is not the same as restarting an app.

HPA points to a nonexistent Deployment

Autoscaler target is wrong

What breaks visually



Production symptom

HPA exists but cannot scale because target object is missing/wrong.

Read events first. Do not guess from pod status only.

```
kubectl get hpa -n app
kubectl describe hpa api-hpa -n app
kubectl get deploy -n app
kubectl get hpa api-hpa -o yaml
```

1 Fault signal
HPA describe says failed to get scale or target not found.

2 Investigate in order
Check scaleTargetRef apiVersion, kind, name, and namespace.

3 Root cause
HPA references an old Deployment name or wrong kind/apiVersion.

4 Repair
Patch or recreate HPA with the correct scaleTargetRef.

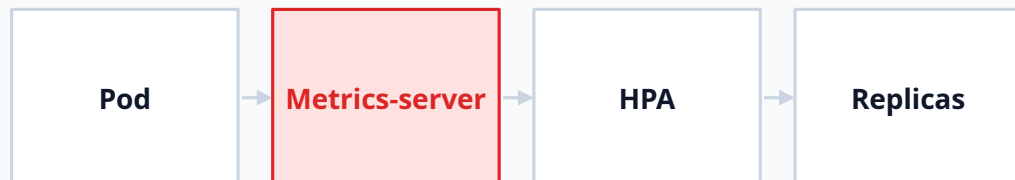
5 Verify recovery
HPA shows REFERENCE deploy/api and current metrics.

6 Lesson
HPA is namespace-scoped and scales exactly the object named in scaleTargetRef.

HPA cannot read metrics

Metrics API missing or broken

What breaks visually



Production symptom
HPA exists but CPU shows <unknown> and replicas do not increase.
Read events first. Do not guess from pod status only.

```
kubectl get hpa -n app
kubectl top pods -n app
kubectl get apiservice v1beta1.metrics.k8s.io
kubectl logs -n kube-system deploy/metrics-server
```

1 Fault signal
HPA TARGETS show <unknown>; kubectl top fails.

2 Investigate in order
Check metrics-server deployment, APIService availability, TLS flags, and resource requests.

3 Root cause
Metrics API unavailable, metrics-server cannot scrape kubelets, or pods have no CPU requests.

4 Repair
Install/fix metrics-server and set CPU requests on target pods.

5 Verify recovery
kubectl top works and HPA shows numeric current/target values.

6 Lesson
HPA needs metrics and requests. Without CPU request, utilization percentage cannot be calculated.

PVC cannot find its StorageClass

Claim stays Pending

What breaks visually



Production symptom
PVC remains Pending because dynamic provisioning cannot happen.
Read events first. Do not guess from pod status only.

```
kubectl get pvc -n app
kubectl describe pvc data-api-0 -n app
kubectl get sc
kubectl get events -n app --sort-by=.lastTimestamp
```

1 Fault signal
PVC Pending; Events mention storageclass not found or no provisioner.

2 Investigate in order
Check storageClassName, default StorageClass, provisioner, accessMode, and capacity.

3 Root cause
Requested StorageClass does not exist or cluster has no default StorageClass.

4 Repair
Create correct StorageClass, fix PVC storageClassName, or bind to pre-created PV.

5 Verify recovery
PVC becomes Bound before pod starts.

6 Lesson
A pod using PVC waits for storage. Storage problems appear before app problems.

Pod references a missing PVC

Volume cannot mount

What breaks visually



Production symptom

**Pod cannot start because its claimName does not exist.
Read events first. Do not guess from pod status only.**

```
kubectl describe pod db-0 -n app
kubectl get pvc -n app
kubectl get deploy/statefulset db -o yaml | grep -A8 claimName
kubectl apply -f pvc.yaml
```

1 Fault signal
Events: persistentvolumeclaim not found.

2 Investigate in order
Check claimName spelling, namespace, StatefulSet volumeClaimTemplates, and PVC status.

3 Root cause
Workload references a PVC that was never created or was deleted.

4 Repair
Create PVC with expected name or update workload claimName.

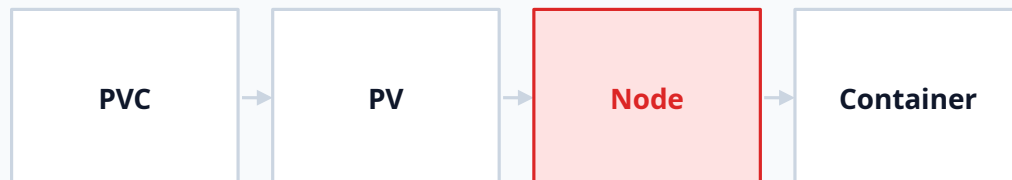
5 Verify recovery
Pod starts volume mount and reaches Running.

6 Lesson
PVC names are namespace-scoped and must match exactly.

PVC is Bound but mount fails

Volume permissions or attach issue

What breaks visually



Production symptom

Storage is bound, but container cannot read/write or volume attach fails.

Read events first. Do not guess from pod status only.

```
kubectl describe pod app-0 -n app
kubectl describe pvc data-app-0 -n app
kubectl describe pv PV_NAME
kubectl logs -n kube-system -l app=csi-controller --tail=80
```

1 Fault signal
MountVolume.SetUp failed, attach timeout, or permission denied inside app.

2 Investigate in order
Check CSI events/logs, access mode, node attach, fsGroup, securityContext, file ownership.

3 Root cause
Volume driver issue, wrong access mode, or filesystem ownership incompatible with app user.

4 Repair
Fix accessMode, StorageClass/CSI, fsGroup/runAsUser, or init chmod/chown if appropriate.

5 Verify recovery
Pod mounts volume and app can write a test file.

6 Lesson
Bound PVC means storage was allocated; it does not guarantee mount permissions are correct.

Deployment selector is immutable

Cannot apply changed selector

What breaks visually



Production symptom
kubectl apply fails because spec.selector cannot be changed.
Read events first. Do not guess from pod status only.

```
kubectl apply -f deployment.yaml
kubectl get deploy nginx -o yaml | grep -A5 selector
kubectl get rs -l app=nginx
kubectl delete deploy nginx # only when safe / planned
```

1 Fault signal
Error: field is immutable for spec.selector.

2 Investigate in order
Compare old selector with new YAML matchLabels and pod template labels.

3 Root cause
A Deployment selector was changed after creation.

4 Repair
Keep original selector, or create a new Deployment name and migrate traffic safely.

5 Verify recovery
Apply succeeds; new pods owned by expected ReplicaSet.

6 Lesson
Deployment selector defines ownership. Kubernetes protects it from unsafe changes.

Rolling update stalls

New ReplicaSet never becomes available

What breaks visually



Production symptom

**Rollout is stuck because new pods fail readiness or crash.
Read events first. Do not guess from pod status only.**

```
kubectl rollout status deploy/api -n app
kubectl describe deploy api -n app
kubectl get rs,pods -l app=api -n app
kubectl rollout undo deploy/api -n app
```

1 Fault signal
ProgressDeadlineExceeded or rollout status keeps waiting.

2 Investigate in order
Check new ReplicaSet pods, Events, readiness, logs, and recent image/config changes.

3 Root cause
New version cannot become Ready so old version remains or capacity is exhausted.

4 Repair
Fix new version or rollback; adjust maxSurge/maxUnavailable only after root cause is known.

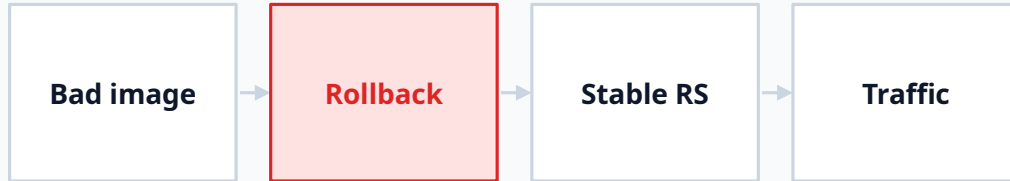
5 Verify recovery
rollout status completes and available replicas meet desired count.

6 Lesson
A rollout problem is usually a pod problem inside the newest ReplicaSet.

Rollback after bad release

Fast recovery while preserving evidence

What breaks visually



Production symptom

**After a release, errors increase and you need safe restoration.
Read events first. Do not guess from pod status only.**

```
kubectl rollout history deploy/api -n app
kubectl rollout undo deploy/api -n app
kubectl rollout status deploy/api -n app
kubectl logs -l app=api -n app --tail=100
```

1 Fault signal
New release causes failed probes, crashes, errors, or latency.

2 Investigate in order
Confirm change window, compare ReplicaSets, collect logs/events before undo.

3 Root cause
Image/config/code change broke health or functionality.

4 Repair
Rollback Deployment, then prepare fixed image/config in a new release.

5 Verify recovery
Old version Ready, error rate drops, no new bad ReplicaSet pods.

6 Lesson
Rollback is repair, not root-cause analysis. Save evidence before deleting pods.

LoadBalancer Service has no external IP

Cloud integration or quota issue

What breaks visually



Production symptom

Service type LoadBalancer remains Pending.

Read events first. Do not guess from pod status only.

```
kubectl get svc api-lb -n app
kubectl describe svc api-lb -n app
kubectl get events -A --sort-by=.lastTimestamp
kubectl logs -n kube-system -l component=cloud-controller-manager --tail=80
```

1 Fault signal
EXTERNAL-IP stays pending for LoadBalancer service.

2 Investigate in order
Check cloud controller, subnet tags, public IP quota, annotations, firewall, and provider limits.

3 Root cause
Cluster cloud integration cannot provision a load balancer.

4 Repair
Fix cloud controller/annotations/subnets/quota or use NodePort/Ingress temporarily.

5 Verify recovery
Service gets external IP/hostname and health checks pass.

6 Lesson
LoadBalancer is not magic inside Kubernetes; the cloud provider must create the real LB.

Ingress returns 404 or default backend

Host/path does not match rule

What breaks visually



Production symptom

Ingress controller is reachable but sends traffic to default backend/404.

Read events first. Do not guess from pod status only.

```
kubectl get ingress -n app
kubectl describe ingress app-ing -n app
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller --tail=80
curl -H "Host: app.example.com" http://LB_IP/
```

1 Fault signal
404 from ingress controller, not from your app.

2 Investigate in order
Check host header, path, ingressClassName, controller logs, service name/port.

3 Root cause
Request host/path does not match ingress rule or ingress is handled by wrong controller.

4 Repair
Fix host/path/pathType, service backend, or ingressClassName.

5 Verify recovery
Curl with correct Host reaches service and app logs receive request.

6 Lesson
Ingress routing depends on HTTP Host/path. Browser IP-only testing often misses host rules.

Ingress TLS or certificate mismatch

HTTPS fails before app gets traffic

What breaks visually



Production symptom

**HTTP may work but HTTPS shows certificate or handshake error.
Read events first. Do not guess from pod status only.**

```
kubectl describe ingress app-ing -n app
kubectl get secret tls-app -n app
openssl s_client -connect app.example.com:443 -servername app.example.com
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller --tail=80
```

1 Fault signal

Browser warns certificate invalid, wrong host, or TLS handshake fails.

2 Investigate in order

Check TLS secret exists, cert SAN/CN, host in ingress, and controller cert logs.

3 Root cause

TLS secret is missing/wrong namespace, certificate does not cover host, or DNS points elsewhere.

4 Repair

Create correct TLS secret and reference it under matching hosts; fix DNS.

5 Verify recovery

openssl/browser shows expected certificate and app receives HTTPS traffic.

6 Lesson

TLS secret name is namespace-scoped and certificate must match the exact hostname.

NetworkPolicy blocks traffic

Pods cannot talk after policy change

What breaks visually



Production symptom

**Service and endpoints are correct, but traffic times out.
Read events first. Do not guess from pod status only.**

```
kubectl get networkpolicy -n app
kubectl describe networkpolicy -n app
kubectl run tmp --rm -it --image=busybox:1.36 -n app -- wget -S http://api-svc
kubectl get pods -n app --show-labels
```

1 Fault signal
Timeouts started after NetworkPolicy or namespace label changes.

2 Investigate in order
Check ingress/egress policies, pod labels, namespace labels, and CNI support.

3 Root cause
Default deny or restrictive policy does not allow required traffic.

4 Repair
Add explicit allow policy for client namespace/pod and destination port.

5 Verify recovery
Test pod can connect; app traffic recovers.

6 Lesson
Once a pod is selected by ingress policy, only allowed traffic can enter it.

RBAC Forbidden

Controller or user cannot access resource

What breaks visually



Production symptom
kubectl or controller logs show forbidden errors.
Read events first. Do not guess from pod status only.

```
kubectl auth can-i get pods -n app
kubectl auth can-i create secrets --as=system:serviceaccount:app:api-sa -n app
kubectl describe rolebinding -n app
kubectl logs deploy/operator -n app
```

1 Fault signal
Error from server (Forbidden) or controller cannot list/watch resources.

2 Investigate in order
Check ServiceAccount, Role/ClusterRole, RoleBinding/ClusterRoleBinding, namespace.

3 Root cause
Identity lacks required verb/resource/scope.

4 Repair
Grant least-privilege Role and bind it to the correct ServiceAccount/user/group.

5 Verify recovery
kubectl auth can-i returns yes; controller errors disappear.

6 Lesson
RBAC answer depends on identity + verb + resource + namespace/scope.

Node pressure causes evictions

Cluster is alive but pods disappear

What breaks visually



Production symptom

**Pods are evicted due to disk, memory, or PID pressure.
Read events first. Do not guess from pod status only.**

```
kubectl describe node NODE
kubectl get events -A --sort-by=.lastTimestamp | grep -i Evicted
kubectl top nodes
kubectl describe pod POD -n app
```

1 Fault signal
Pod status Evicted; node conditions show MemoryPressure/DiskPressure/PIDPressure.

2 Investigate in order
Check node conditions, allocatable, image garbage, ephemeral storage, logs, and daemon usage.

3 Root cause
Node resources crossed eviction thresholds.

4 Repair
Free disk, reduce logs/temp files, set ephemeral-storage requests/limits, add capacity, move workloads.

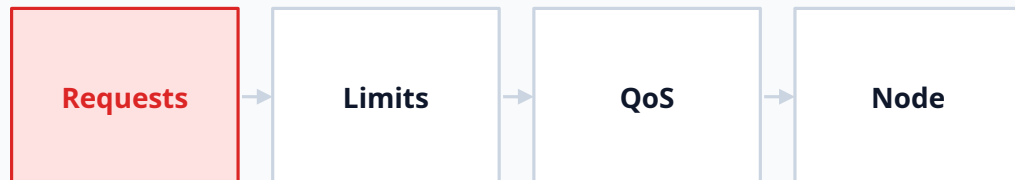
5 Verify recovery
Node pressure condition clears and evictions stop.

6 Lesson
Production capacity includes ephemeral storage and logs, not just CPU/RAM.

Requests and limits are mis-sized

Performance or scheduling pain

What breaks visually



Production symptom

Too high requests block scheduling; too low limits cause throttling/OOM.

Read events first. Do not guess from pod status only.

```
kubectl top pod -n app
kubectl describe pod api-abc -n app | grep -A8 Limits
kubectl describe node NODE | grep -A8 "Allocated resources"
kubectl get pod api-abc -o jsonpath="{.status.qosClass}"
```

1 Fault signal
Pending pods, CPU throttling, OOMKills, or unstable latency.

2 Investigate in order
Compare requests/limits with real usage percentiles and SLO needs.

3 Root cause
Resource settings do not match real workload behavior.

4 Repair
Set realistic requests; use limits carefully; scale replicas; test under load.

5 Verify recovery
Pods schedule reliably and latency/restarts improve.

6 Lesson
Requests protect scheduling. Limits enforce ceilings. Bad values cause production incidents.

Crash only under load

The app passes basic health but fails at traffic

What breaks visually



Production symptom

**Everything looks fine at idle, then failures appear during load.
Read events first. Do not guess from pod status only.**

```
kubectl top pods -n app --containers
kubectl logs -l app=api -n app --tail=200
kubectl describe hpa api-hpa -n app
kubectl get events -n app --sort-by=.lastTimestamp
```

1 Fault signal
Spikes cause 5xx, latency, restarts, or failed probes.

2 Investigate in order
Check CPU throttling, memory growth, dependency saturation, HPA delay, probe timeouts.

3 Root cause
Capacity, timeout, or dependency bottleneck appears only under real load.

4 Repair
Tune resources, replicas, HPA, connection pools, timeouts, and readiness behavior.

5 Verify recovery
Run controlled load test; watch metrics, restarts, and error rate.

6 Lesson
A healthy idle pod can still be a weak production pod.

Namespace or context mistake

You troubleshoot the wrong cluster or namespace

What breaks visually



Production symptom

Commands show missing resources because you are not where you think you are.

Read events first. Do not guess from pod status only.

```
kubectl config current-context
kubectl config get-contexts
kubectl config view --minify --output "jsonpath={..namespace}"
kubectl get ns
```

1 Fault signal
Object not found, but teammate sees it; changes do nothing.

2 Investigate in order
Check current context, namespace, cluster server, and kubectl aliases.

3 Root cause
Using wrong kubeconfig context or default namespace.

4 Repair
Switch context/namespace explicitly; include -n in production commands.

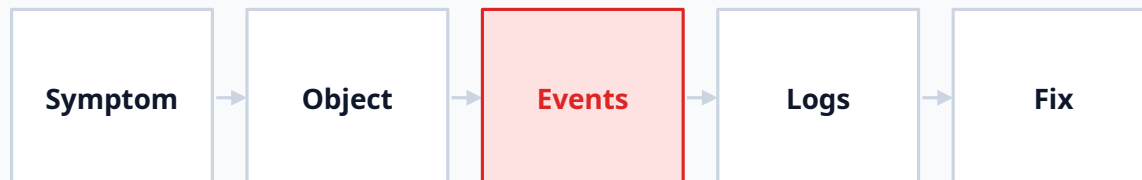
5 Verify recovery
Commands show expected resources and events.

6 Lesson
Before deleting or patching, confirm context and namespace. This prevents accidental outages.

Production investigation order

A clean decision tree for any Kubernetes incident

What breaks visually



Production symptom

When confused, follow this order instead of random commands. Read events first. Do not guess from pod status only.

```
kubectl get all -n app
kubectl describe pod POD -n app
kubectl logs POD -n app --previous
kubectl get svc,endpoints,ingress,pvc,hpa -n app
kubectl get events -n app --sort-by=.lastTimestamp
```

1 Fault signal
You see downtime but do not yet know the layer.

2 Investigate in order
1 context/ns → 2 inventory → 3 affected object → 4 events → 5 logs → 6 network/storage/metrics.

3 Root cause
Most incidents are one broken link in the request path or workload lifecycle.

4 Repair
Fix the smallest proven cause, not every suspicious thing.

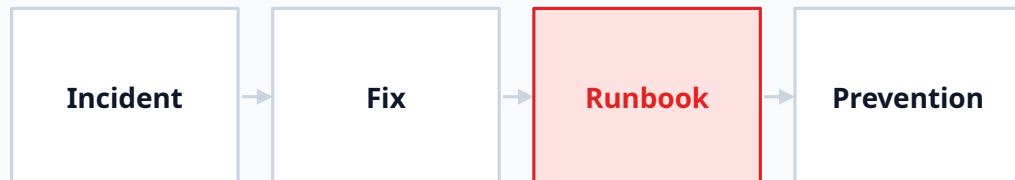
5 Verify recovery
Re-check the exact user path and the Kubernetes object status.

6 Lesson
Troubleshooting is a sequence. Evidence first, YAML change second.

Keep the lesson after recovery

Turn incidents into reusable runbooks

What breaks visually



Production symptom

**Do not waste a solved problem. Convert it into a checklist.
Read events first. Do not guess from pod status only.**

```
mkdir -p runbooks/k8s
cat > runbooks/k8s/imagepullbackoff.md
# Symptom / Root cause / Commands / Fix / Verify / Prevention
git add runbooks && git commit -m "Add k8s incident runbook"
```

1 Fault signal

Same class of incident repeats because knowledge stays in chat or memory.

2 Investigate in order

Collect commands, outputs, root cause, exact fix, verification, and prevention.

3 Root cause

No shared runbook, weak precheck, or missing alert.

4 Repair

Write a short incident note, add precheck to CI/CD, and update monitors.

5 Verify recovery

Next engineer can follow the note without guessing.

6 Lesson

The best production teams improve the system after the fire is out.

One-page Kubernetes troubleshooting checklist

K8s prod runbook · 43

Print this mental model for labs and interviews

What breaks visually



Production symptom

**A short checklist covers most beginner-to-production incidents.
Read events first. Do not guess from pod status only.**

```
1) kubectl config current-context
2) kubectl get pods -A
3) kubectl describe failed object
4) kubectl logs --previous when restarting
5) verify service endpoints and probes
```

1 Fault signal
Something is down, slow, Pending, NotReady, or restarting.

2 Investigate in order
Context → namespace → object status → events → logs → config/secrets → service/endpoints → resources/storage.

3 Root cause
Find the first broken Kubernetes promise: schedule, pull, start, ready, route, scale, or persist.

4 Repair
Change one thing at a time and prefer reversible fixes.

5 Verify recovery
User path works, rollout complete, events quiet, metrics normal.

6 Lesson
Mastering Kubernetes means mastering its failure messages and object relationships.

Pattern: fault → investigate in order → root cause → repair → verify recovery → keep the lesson